

Python Beginner's Guide

A Simple, Friendly Introduction to Python Programming

Compiled with Claude

2026

Contents

Chapter 1: Introduction to Python	2
What is Python?	2
Installing Python	2
Writing Your First Program	2
Chapter 2: Variables and Data Types	3
What is a Variable?	3
Naming Rules for Variables	3
Basic Data Types	3
Checking a Variable's Type	3
Type Conversion	4
Basic Input and Output	4
Chapter 3: Operators	5
Arithmetic Operators	5
Comparison Operators	5
Logical Operators	5
Assignment Operators	5
Chapter 4: Control Flow (if-else)	6
The if Statement	6
if-else	6
if-elif-else	6
Nested if	6
Chapter 5: Loops	7
The for Loop	7
range() Variations	7
The while Loop	7
break and continue	7
Loop with else	8
Chapter 6: Strings	9
Creating Strings	9

String Indexing and Slicing	9
Common String Methods	9
String Formatting	9
String Concatenation	10
Chapter 7: Lists	11
What is a List?	11
Accessing List Items	11
Modifying Lists	11
Looping Through a List	11
List Comprehension	11
Chapter 8: Tuples	13
What is a Tuple?	13
Accessing Tuple Items	13
Why Use Tuples?	13
Tuple Unpacking	13
Chapter 9: Dictionaries	14
What is a Dictionary?	14
Accessing Values	14
Modifying a Dictionary	14
Looping Through a Dictionary	14
Chapter 10: Sets	15
What is a Set?	15
Set Operations	15
Chapter 11: Functions	16
What is a Function?	16
Functions with Parameters	16
Functions with Return Values	16
Default Parameter Values	16
Keyword Arguments	16
*args and **kwargs	17
Variable Scope	17
Lambda Functions	17
Chapter 12: Object-Oriented Programming (OOP)	19
What is OOP?	19
Classes and Objects	19
Instance vs Class Variables	19
Methods	19
Inheritance	20
The super() Function	20
Encapsulation	21
Polymorphism	21
Chapter 13: File Handling	22

Opening and Reading Files	22
Using with (Recommended)	22
Reading Line by Line	22
Writing to Files	22
Appending to Files	22
File Modes Summary	22
Chapter 14: Exception Handling	23
Why Handle Exceptions?	23
try-except	23
try-except-else-finally	23
Catching Any Exception	23
Raising Your Own Exceptions	23
Chapter 15: Modules and Packages	25
What is a Module?	25
Importing Specific Items	25
Useful Built-in Modules	25
Installing External Packages with pip	25
Chapter 16: A Gentle Look at Advanced Topics	26
List, Dict, and Set Comprehensions	26
Generators	26
Decorators	26
Working with JSON	27
Chapter 17: Mini Projects to Practice	28
Project 1: Simple Calculator	28
Project 2: To-Do List (in-memory)	28
Project 3: Number Guessing Game	29
Challenge: Extend These Projects	30
Chapter 18: Common Errors and How to Fix Them	31
SyntaxError	31
IndentationError	31
NameError	31
TypeError	31
IndexError	32
KeyError	32
ZeroDivisionError	32
Debugging Tips	33
Chapter 19: Practice Exercises	34
Exercise 1: Variables and Data Types	34
Exercise 2: Conditionals	34
Exercise 3: Loops	34
Exercise 4: Strings	34
Exercise 5: Lists	35
Exercise 6: Dictionaries	35

Exercise 7: Functions	36
Exercise 8: OOP	36
Exercise 9: File Handling	36
Exercise 10: Exception Handling	37
Chapter 20: Important Questions and Answers — Basics	38
Q1. What is Python? What are its key features?	38
Q2. What is the difference between Python 2 and Python 3?	38
Q3. Is Python compiled or interpreted?	38
Q4. What is PEP 8?	38
Q5. What is the difference between is and ==?	39
Q6. What are mutable and immutable data types?	39
Q7. What is dynamic typing?	39
Q8. What is the difference between list and tuple?	39
Q9. What is the difference between append() and extend() in lists?	39
Q10. What is the difference between shallow copy and deep copy?	40
Chapter 21: Important Questions and Answers — Control Flow and Loops	41
Q1. What is the difference between break, continue, and pass?	41
Q2. Why does Python use indentation instead of curly braces?	41
Q3. What is the difference between for and while loops?	41
Q4. Can a while loop have an else block? What does it do?	42
Q5. What does the range() function do, and what are its parameters?	42
Q6. How do you loop through a list with both index and value?	42
Q7. What is a nested loop? Give an example.	42
Q8. What will be the output of this code, and why?	42
Chapter 22: Important Questions and Answers — Functions	44
Q1. What is the difference between a parameter and an argument?	44
Q2. What is the difference between *args and **kwargs?	44
Q3. What is a recursive function? Give an example.	44
Q4. What is the difference between return and print() inside a function?	44
Q5. What is a default argument? What is a common pitfall?	45
Q6. What is variable scope? Explain local, global, and the global keyword.	45
Q7. What is a lambda function? When would you use one?	46
Q8. What is the difference between map(), filter(), and reduce()?	46
Chapter 23: Important Questions and Answers — OOP	47
Q1. What are the four pillars of OOP?	47
Q2. What is the difference between a class and an object?	47
Q3. What is self in Python classes?	47
Q4. What is the difference between instance variables and class variables?	47
Q5. What is method overriding? Give an example.	48
Q6. What does super() do?	48
Q7. What is the difference between single, multiple, and multilevel inheritance?	48
Q8. What is encapsulation? How is it implemented in Python?	49
Q9. What is polymorphism? Give a practical example.	49
Q10. What is the difference between an abstract class and a regular class?	50

Chapter 24: Important Questions and Answers — Data Structures	51
Q1. What is the difference between a list, tuple, set, and dictionary?	51
Q2. How do you remove duplicates from a list?	51
Q3. How do you merge two dictionaries?	51
Q4. What is list slicing? Give examples.	52
Q5. What is a set, and what operations does it support?	52
Q6. How do you sort a list of dictionaries by a specific key?	52
Q7. What is the time complexity of common list and dictionary operations? .	52
Chapter 25: Important Questions and Answers — Exceptions and Modules	54
Q1. What is the difference between an error and an exception?	54
Q2. What is the purpose of finally in exception handling?	54
Q3. What is the difference between except Exception and a specific excep- tion like except ValueError?	54
Q4. Can you have multiple except blocks for one try? Show an example. . .	54
Q5. How do you create and raise a custom exception?	55
Q6. What is the difference between a module and a package?	55
Q7. What is the difference between import module and from module import function?	55
Q8. What is pip, and how do you use it?	56
Chapter 26: Frequently Asked Interview-Style Questions	57
Q1. How is memory managed in Python?	57
Q2. What is the Global Interpreter Lock (GIL)?	57
Q3. What are Python's built-in data types?	57
Q4. What is the difference between == and .equals()-style comparisons across types?	57
Q5. What is a generator, and how is it different from a normal function? . .	57
Q6. What is a decorator?	58
Q7. What is the difference between deepcopy and assignment (=)?	58
Q8. What is duck typing in Python?	59
Q9. What is the difference between __init__ and __new__?	59
Q10. What does the __name__ == "__main__" check do?	59
Chapter 27: Rapid-Fire Question Bank (One-Line Answers)	60
Chapter 28: Where to Go From Here	63
Quick Reference: Common Built-in Functions	63

Chapter 1: Introduction to Python

What is Python?

Python is a popular, easy-to-read programming language used for web development, data analysis, automation, artificial intelligence, and much more. It was created by Guido van Rossum and first released in 1991.

Python is loved by beginners because:

- Its syntax reads almost like plain English.
- It requires fewer lines of code than many other languages.
- It has a huge community and tons of free libraries.
- It runs on Windows, Mac, and Linux.

Installing Python

1. Visit **python.org/downloads**
2. Download the latest version for your operating system.
3. During installation (Windows), check the box that says **“Add Python to PATH”**.
4. Verify installation by opening a terminal and typing:

```
python --version
```

Writing Your First Program

Open any text editor (or VS Code), create a file called `hello.py`, and type:

```
print("Hello, World!")
```

Run it from the terminal:

```
python hello.py
```

Output:

Hello, World!

Congratulations — you just wrote your first Python program!

Chapter 2: Variables and Data Types

What is a Variable?

A variable is a named container that stores a value in memory. In Python, you don't need to declare a type — Python figures it out automatically.

```
name = "Arun"
age = 25
height = 5.9
is_student = True
```

Naming Rules for Variables

- Must start with a letter or underscore (not a number).
- Can contain letters, numbers, and underscores.
- Case-sensitive (age and Age are different).
- Cannot use Python keywords (like print, if, for) as variable names.

```
# Valid
student_name = "Kavi"
_total = 100

# Invalid
2nd_value = 5    # Error: starts with a number
```

Basic Data Types

Type	Example	Description
int	10	Whole numbers
float	10.5	Decimal numbers
str	"hello"	Text
bool	True / False	Logical value
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{"key": "value"}	Key-value pairs
set	{1, 2, 3}	Unordered, unique values

Checking a Variable's Type

```
x = 10
print(type(x))    # <class 'int'>

y = "Python"
print(type(y))    # <class 'str'>
```

Type Conversion

You can convert between types using built-in functions:

```
age = "25"
age_number = int(age)      # string to integer
price = float("99.99")    # string to float
text = str(100)           # integer to string
```

Basic Input and Output

```
name = input("Enter your name: ")
print("Hello, " + name + "!")
```

The `input()` function always returns a string, so convert it if you need a number:

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)
```


Chapter 3: Operators

Arithmetic Operators

```
a = 10
b = 3

print(a + b)    # 13  Addition
print(a - b)    # 7   Subtraction
print(a * b)    # 30  Multiplication
print(a / b)    # 3.333... Division (always returns float)
print(a // b)   # 3   Floor division (drops the decimal)
print(a % b)    # 1   Modulus (remainder)
print(a ** b)   # 1000 Exponent (power)
```

Comparison Operators

```
print(5 == 5)   # True
print(5 != 3)   # True
print(5 > 3)    # True
print(5 < 3)    # False
print(5 >= 5)   # True
print(5 <= 4)   # False
```

Logical Operators

```
x = True
y = False

print(x and y)  # False - both must be True
print(x or y)   # True  - at least one must be True
print(not x)    # False - reverses the value
```

Assignment Operators

```
x = 5
x += 3    # x = x + 3 -> 8
x -= 2    # x = x - 2 -> 6
x *= 4    # x = x * 4 -> 24
x /= 2    # x = x / 2 -> 12.0
```

Chapter 4: Control Flow (if-else)

The if Statement

```
age = 20

if age >= 18:
    print("You are an adult.")
```

Notice the **colon (:)** and the **indentation** (4 spaces). Python uses indentation instead of curly braces {} to define blocks of code.

if-else

```
age = 15

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")
```

if-elif-else

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Nested if

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Too young")
```

Chapter 5: Loops

The for Loop

Used when you want to repeat an action a known number of times, or go through items in a collection.

```
for i in range(5):  
    print(i)  
# Output: 0 1 2 3 4
```

range(5) generates numbers from 0 to 4 (5 is excluded).

```
fruits = ["apple", "banana", "cherry"]  
  
for fruit in fruits:  
    print(fruit)
```

range() Variations

```
range(5)          # 0,1,2,3,4  
range(2, 6)       # 2,3,4,5  
range(0, 10, 2)   # 0,2,4,6,8 (step of 2)
```

The while Loop

Used when you want to repeat something until a condition becomes false.

```
count = 0  
  
while count < 5:  
    print(count)  
    count += 1
```

Be careful: if the condition never becomes false, you get an infinite loop.

break and continue

```
for i in range(10):  
    if i == 5:  
        break          # stops the loop completely  
    print(i)  
  
for i in range(5):  
    if i == 2:  
        continue       # skips this iteration, goes to next  
    print(i)
```

Loop with else

```
for i in range(3):  
    print(i)  
else:  
    print("Loop finished normally")
```

The else block runs only if the loop completes without a break.

Chapter 6: Strings

Creating Strings

```
name = "Python"
greeting = 'Hello'
multiline = """This is
a multiline string"""
```

String Indexing and Slicing

```
word = "Python"

print(word[0])      # P (first character)
print(word[-1])     # n (last character)
print(word[0:3])    # Pyt (slice from index 0 to 2)
print(word[:3])     # Pyt
print(word[3:])     # hon
print(word[::-1])   # nohtyP (reversed)
```

Common String Methods

```
text = " Hello Python "
```



```
print(text.strip())      # "Hello Python" (removes spaces)
print(text.upper())      # " HELLO PYTHON "
print(text.lower())      # " hello python "
print(text.replace("Hello", "Hi"))
print(text.split())      # ['Hello', 'Python']
print(len(text))         # length of string
print("Python" in text)  # True (membership check)
```

String Formatting

```
name = "Kavi"
age = 22
```



```
# f-strings (recommended, Python 3.6+)
print(f"My name is {name} and I am {age} years old.")
```



```
# .format() method
print("My name is {} and I am {} years old.".format(name, age))
```



```
# % formatting (older style)
print("My name is %s and I am %d years old." % (name, age))
```

String Concatenation

```
first = "Python"
second = "Programming"

combined = first + " " + second
print(combined)    # Python Programming

repeated = "Ha" * 3
print(repeated)    # HaHaHa
```

Chapter 7: Lists

What is a List?

A list is an ordered, changeable collection that can hold items of any type.

```
fruits = ["apple", "banana", "cherry"]
mixed = [1, "two", 3.0, True]
```

Accessing List Items

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])    # apple
print(fruits[-1])   # cherry (last item)
print(fruits[1:3])  # ['banana', 'cherry']
```

Modifying Lists

```
fruits = ["apple", "banana", "cherry"]

fruits[1] = "mango"           # change an item
fruits.append("orange")       # add to the end
fruits.insert(1, "grape")     # insert at position
fruits.remove("apple")        # remove by value
fruits.pop()                  # remove last item
fruits.pop(0)                  # remove item at index 0
fruits.sort()                  # sort alphabetically
fruits.reverse()              # reverse order
print(len(fruits))            # number of items
```

Looping Through a List

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

# With index
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

List Comprehension

A shorter way to create lists:

```
squares = [x**2 for x in range(5)]  
print(squares)    # [0, 1, 4, 9, 16]  
  
evens = [x for x in range(10) if x % 2 == 0]  
print(evens)      # [0, 2, 4, 6, 8]
```


Chapter 8: Tuples

What is a Tuple?

A tuple is like a list, but **unchangeable (immutable)** once created. Use tuples for data that should not change.

```
coordinates = (10, 20)
colors = ("red", "green", "blue")
```

Accessing Tuple Items

```
colors = ("red", "green", "blue")

print(colors[0])    # red
print(colors[-1])   # blue
print(colors[0:2])  # ('red', 'green')
```

Why Use Tuples?

- Faster than lists.
- Protects data from accidental changes.
- Can be used as dictionary keys (lists cannot).

```
# This will cause an ERROR because tuples can't be changed:
colors = ("red", "green", "blue")
# colors[0] = "yellow" # TypeError!
```

Tuple Unpacking

```
point = (4, 5)
x, y = point
print(x, y)    # 4 5
```

Chapter 9: Dictionaries

What is a Dictionary?

A dictionary stores data as **key-value pairs**. Each key must be unique.

```
student = {  
    "name": "Arun",  
    "age": 21,  
    "course": "Python"  
}
```

Accessing Values

```
print(student["name"])          # Arun  
print(student.get("age"))       # 21  
print(student.get("grade", "N/A")) # N/A (default if key missing)
```

Modifying a Dictionary

```
student["age"] = 22             # update value  
student["grade"] = "A"          # add new key-value pair  
del student["course"]           # remove a key  
student.pop("grade")            # another way to remove
```

Looping Through a Dictionary

```
for key in student:  
    print(key, ":", student[key])  
  
for key, value in student.items():  
    print(key, "->", value)  
  
print(student.keys())           # all keys  
print(student.values())         # all values
```

Chapter 10: Sets

What is a Set?

A set is an unordered collection of **unique** items — duplicates are automatically removed.

```
numbers = {1, 2, 3, 3, 2, 1}
print(numbers)    # {1, 2, 3}
```

Set Operations

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a.union(b))      # {1, 2, 3, 4}
print(a.intersection(b)) # {2, 3}
print(a.difference(b))  # {1}

a.add(5)               # add an item
a.remove(1)            # remove an item
```

Sets are useful when you need to quickly check membership or remove duplicate values from a list:

```
my_list = [1, 2, 2, 3, 3, 3]
unique_items = list(set(my_list))
print(unique_items)    # [1, 2, 3]
```

Chapter 11: Functions

What is a Function?

A function is a reusable block of code that performs a specific task. Functions help avoid repeating code.

```
def greet():  
    print("Hello!")  
  
greet()    # calling the function
```

Functions with Parameters

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Kavi")  
greet("Arun")
```

Functions with Return Values

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)    # 8
```

Default Parameter Values

```
def greet(name="Guest"):  
    print(f"Hello, {name}!")  
  
greet()           # Hello, Guest!  
greet("Priya")    # Hello, Priya!
```

Keyword Arguments

```
def describe(name, age):  
    print(f"{name} is {age} years old")  
  
describe(age=25, name="Ravi")    # order doesn't matter
```

***args and **kwargs**

Use *args for a variable number of positional arguments, and **kwargs for a variable number of keyword arguments.

```
def add_all(*numbers):  
    return sum(numbers)  
  
print(add_all(1, 2, 3, 4))    # 10  
  
def show_info(**details):  
    for key, value in details.items():  
        print(key, ":", value)  
  
show_info(name="Kavi", age=22, city="Chennai")
```

Variable Scope

```
x = 10    # global variable  
  
def my_function():  
    x = 5    # local variable (different from global x)  
    print(x)  
  
my_function()    # 5  
print(x)         # 10
```

Use the global keyword to modify a global variable inside a function:

```
counter = 0  
  
def increment():  
    global counter  
    counter += 1  
  
increment()  
print(counter)    # 1
```

Lambda Functions

A lambda is a small, anonymous, one-line function.

```
square = lambda x: x ** 2  
print(square(5))    # 25  
  
add = lambda a, b: a + b  
print(add(3, 4))    # 7
```

Lambdas are often used with functions like map(), filter(), and sorted():

```
numbers = [1, 2, 3, 4, 5]

doubled = list(map(lambda x: x * 2, numbers))
print(doubled)      # [2, 4, 6, 8, 10]

evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)        # [2, 4]
```

Chapter 12: Object-Oriented Programming (OOP)

What is OOP?

Object-Oriented Programming organizes code around **objects**, which combine data (attributes) and behavior (methods). Python is fully object-oriented.

Classes and Objects

A **class** is a blueprint. An **object** is an instance created from that blueprint.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Creating objects (instances)
dog1 = Dog("Rocky", 3)
dog2 = Dog("Bruno", 5)

dog1.bark()    # Rocky says Woof!
print(dog2.age) # 5
```

`__init__` is a special method called a **constructor** — it runs automatically when a new object is created. `self` refers to the current object.

Instance vs Class Variables

```
class Dog:
    species = "Canine"    # class variable, shared by all instances

    def __init__(self, name):
        self.name = name    # instance variable, unique to each object

dog1 = Dog("Rocky")
dog2 = Dog("Bruno")

print(dog1.species)    # Canine
print(dog2.species)    # Canine
print(dog1.name)       # Rocky
```

Methods

```

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

    def circumference(self):
        return 2 * 3.14 * self.radius

c = Circle(5)
print(c.area())           # 78.5
print(c.circumference())  # 31.4

```

Inheritance

Inheritance lets a class (child) reuse code from another class (parent).

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Cat(Animal):           # Cat inherits from Animal
    def speak(self):        # method overriding
        print(f"{self.name} says Meow")

class Bird(Animal):
    pass                    # uses Animal's speak() method as is

cat = Cat("Whiskers")
bird = Bird("Tweety")

cat.speak()                # Whiskers says Meow
bird.speak()               # Tweety makes a sound

```

The super() Function

super() lets a child class call a method from its parent class.

```

class Animal:
    def __init__(self, name):
        self.name = name

class Cat(Animal):
    def __init__(self, name, color):

```



```

        super().__init__(name)    # call parent's __init__
        self.color = color

cat = Cat("Whiskers", "black")
print(cat.name, cat.color)    # Whiskers black

```

Encapsulation

Encapsulation means restricting direct access to some of an object's data. In Python, a leading underscore signals "internal use," and a double underscore makes it harder to access from outside.

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # "private" variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())    # 1500

```

Polymorphism

Polymorphism means different classes can be used through the same interface, each responding to the same method call in its own way (as shown by `speak()` in the Cat/Bird example above).

Chapter 13: File Handling

Opening and Reading Files

```
file = open("data.txt", "r")    # "r" = read mode
content = file.read()
print(content)
file.close()
```

Using with (Recommended)

The with statement automatically closes the file, even if an error occurs.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# file is automatically closed here
```

Reading Line by Line

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Writing to Files

```
with open("output.txt", "w") as file:    # "w" = write mode (overwrites)
    file.write("Hello, Python!\n")
    file.write("This is a new line.")
```

Appending to Files

```
with open("output.txt", "a") as file:    # "a" = append mode (adds to end)
    file.write("\nThis line is added.")
```

File Modes Summary

Mode	Meaning
"r"	Read (default, file must exist)
"w"	Write (creates new or overwrites)
"a"	Append (adds to end of file)
"r+"	Read and write

Chapter 14: Exception Handling

Why Handle Exceptions?

Errors happen — a file may be missing, or a user might enter the wrong type of input. Exception handling lets your program respond gracefully instead of crashing.

try-except

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("You can't divide by zero!")
```

try-except-else-finally

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Division successful:", result)
finally:
    print("This always runs, error or not")
```

Catching Any Exception

```
try:
    risky_code()
except Exception as e:
    print("Something went wrong:", e)
```

Raising Your Own Exceptions

```
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    check_age(-5)
```

```
except ValueError as e:  
    print(e)    # Age cannot be negative
```

Chapter 15: Modules and Packages

What is a Module?

A module is simply a Python file containing functions, classes, or variables that you can reuse in other files.

```
# my_module.py
def greet(name):
    return f"Hello, {name}!"
```

```
# main.py
import my_module

print(my_module.greet("Kavi"))
```

Importing Specific Items

```
from my_module import greet

print(greet("Arun"))
```

Useful Built-in Modules

```
import math
print(math.sqrt(16))      # 4.0
print(math.pi)           # 3.14159...

import random
print(random.randint(1, 10))  # random number between 1 and 10

import datetime
print(datetime.date.today())  # today's date

import os
print(os.getcwd())          # current working directory
```

Installing External Packages with pip

Python's package manager, pip, lets you install third-party libraries:

```
pip install requests
pip install pandas
```

```
import requests
response = requests.get("https://api.example.com")
print(response.status_code)
```

Chapter 16: A Gentle Look at Advanced Topics

These topics are a little more advanced, but useful to know about even as a beginner — you'll meet them often when reading other people's code.

List, Dict, and Set Comprehensions

```
squares = [x**2 for x in range(6)]           # list comprehension
square_map = {x: x**2 for x in range(6)}      # dict comprehension
unique_lengths = {len(word) for word in ["hi", "hey", "hello"]} # set comprehension
```

Generators

A generator is like a function, but instead of returning one value with `return`, it can yield many values one at a time — useful for big datasets, since it doesn't store everything in memory at once.

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

for number in count_up_to(5):
    print(number)    # 1 2 3 4 5
```

Decorators

A decorator is a function that adds extra behavior to another function, without changing its code.

```
def my_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Before the function runs
# Hello!
# After the function runs
```

Working with JSON

JSON is a common format for exchanging data, especially with web APIs.

```
import json

data = {"name": "Kavi", "age": 22}

json_string = json.dumps(data)    # Python dict -> JSON string
print(json_string)

parsed_back = json.loads(json_string)  # JSON string -> Python dict
print(parsed_back["name"])
```

Chapter 17: Mini Projects to Practice

Reading is good, but the best way to learn Python is by building small projects. Here are three beginner-friendly projects that use everything you've learned so far.

Project 1: Simple Calculator

```
def calculator():
    print("Simple Calculator")
    print("Operations: +, -, *, /")

    num1 = float(input("Enter first number: "))
    operator = input("Enter operator (+, -, *, /): ")
    num2 = float(input("Enter second number: "))

    if operator == "+":
        result = num1 + num2
    elif operator == "-":
        result = num1 - num2
    elif operator == "*":
        result = num1 * num2
    elif operator == "/":
        if num2 == 0:
            print("Error: Cannot divide by zero")
            return
        result = num1 / num2
    else:
        print("Invalid operator")
        return

    print(f"Result: {result}")

calculator()
```

Concepts used: functions, input/output, if-elif-else, type conversion, exception avoidance (checking for zero).

Project 2: To-Do List (in-memory)

```
def todo_app():
    tasks = []

    while True:
        print("\n1. Add task")
        print("2. View tasks")
        print("3. Remove task")
        print("4. Exit")
```



```

choice = input("Choose an option: ")

if choice == "1":
    task = input("Enter task: ")
    tasks.append(task)
    print("Task added!")

elif choice == "2":
    if not tasks:
        print("No tasks yet.")
    else:
        for index, task in enumerate(tasks, start=1):
            print(f"{index}. {task}")

elif choice == "3":
    if not tasks:
        print("No tasks to remove.")
    else:
        for index, task in enumerate(tasks, start=1):
            print(f"{index}. {task}")
        num = int(input("Enter task number to remove: "))
        if 1 <= num <= len(tasks):
            removed = tasks.pop(num - 1)
            print(f"Removed: {removed}")
        else:
            print("Invalid task number")

elif choice == "4":
    print("Goodbye!")
    break

else:
    print("Invalid choice, try again.")

todo_app()

```

Concepts used: lists, while loop, functions, enumerate, user input, break.

Project 3: Number Guessing Game

```

import random

def guessing_game():
    secret_number = random.randint(1, 100)
    attempts = 0
    max_attempts = 7

```

```

print("I'm thinking of a number between 1 and 100.")
print(f"You have {max_attempts} attempts to guess it.")

while attempts < max_attempts:
    guess = int(input("Your guess: "))
    attempts += 1

    if guess < secret_number:
        print("Too low!")
    elif guess > secret_number:
        print("Too high!")
    else:
        print(f"Correct! You guessed it in {attempts} attempts.")
        return

print(f"Out of attempts! The number was {secret_number}.")

guessing_game()

```

Concepts used: random module, while loop, if-elif-else, functions.

Challenge: Extend These Projects

Try adding these features yourself:

- **Calculator:** Add support for exponents (**) and modulus (%).
- **To-Do List:** Save tasks to a file so they aren't lost when the program closes (see Chapter 13 on File Handling).
- **Guessing Game:** Let the player choose the difficulty (number range and attempts).

Chapter 18: Common Errors and How to Fix Them

Every programmer — beginner or expert — runs into errors constantly. Learning to read error messages is a superpower. Here are the most common ones you'll see as a beginner.

SyntaxError

Happens when Python can't understand your code structure — usually a missing colon, bracket, or quote.

```
if x > 5
    print("big")
```

SyntaxError: expected ':'

Fix: Add the missing colon after the condition.

```
if x > 5:
    print("big")
```

IndentationError

Python uses indentation to define code blocks. Mixing tabs and spaces, or forgetting to indent, causes this error.

```
def greet():
print("Hello")
```

IndentationError: expected an indented block

Fix: Indent the line inside the function with 4 spaces.

```
def greet():
    print("Hello")
```

NameError

Happens when you use a variable that hasn't been defined yet (often a typo).

```
print(naem)
```

NameError: name 'naem' is not defined

Fix: Check spelling, or make sure the variable was assigned before use.

TypeError

Happens when an operation is used on the wrong data type.

```
age = "25"
print(age + 5)
```

TypeError: can only concatenate str (not "int") to str

Fix: Convert types before combining them.

```
age = int("25")
print(age + 5)    # 30
```

IndexError

Happens when you try to access a list index that doesn't exist.

```
fruits = ["apple", "banana"]
print(fruits[5])
```

IndexError: list index out of range

Fix: Check the list length before accessing an index, or use a valid index.

```
if len(fruits) > 5:
    print(fruits[5])
else:
    print("Index out of range")
```

KeyError

Happens when you try to access a dictionary key that doesn't exist.

```
student = {"name": "Kavi"}
print(student["age"])
```

KeyError: 'age'

Fix: Use `.get()` with a default value instead of direct indexing.

```
print(student.get("age", "Not specified"))
```

ZeroDivisionError

Happens when you divide by zero.

```
print(10 / 0)
```

ZeroDivisionError: division by zero

Fix: Check the denominator before dividing.

```
denominator = 0
if denominator != 0:
    print(10 / denominator)
else:
    print("Cannot divide by zero")
```

Debugging Tips

1. **Read the error message from the bottom up** — the last line usually tells you the exact problem.
2. **Check the line number** mentioned in the error — that's usually where the issue is (or just above it).
3. **Use `print()` statements** to check variable values at different points in your code.
4. **Search the exact error message** online — chances are someone else has hit the same error.
5. **Take a break** — a fresh pair of eyes often spots the bug faster.

Chapter 19: Practice Exercises

Try solving these on your own before checking the solutions. This is where real learning happens!

Exercise 1: Variables and Data Types

Question: Write a program that asks for the user's name and birth year, then calculates and prints their age (assume the current year is 2026).

Solution:

```
name = input("Enter your name: ")
birth_year = int(input("Enter your birth year: "))

current_year = 2026
age = current_year - birth_year

print(f"Hello {name}, you are {age} years old.")
```

Exercise 2: Conditionals

Question: Write a program that checks if a number is positive, negative, or zero.

Solution:

```
number = float(input("Enter a number: "))

if number > 0:
    print("The number is positive.")
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Exercise 3: Loops

Question: Print the multiplication table of a number entered by the user (1 to 10).

Solution:

```
number = int(input("Enter a number: "))

for i in range(1, 11):
    print(f"{number} x {i} = {number * i}")
```

Exercise 4: Strings

Question: Write a program that checks if a given word is a palindrome (reads the same backward as forward).

Solution:

```
word = input("Enter a word: ").lower()

if word == word[::-1]:
    print(f"{word} is a palindrome.")
else:
    print(f"{word} is not a palindrome.")
```

Exercise 5: Lists

Question: Given a list of numbers, find the largest and smallest values without using `max()` or `min()`.

Solution:

```
numbers = [23, 5, 67, 12, 89, 3, 45]

largest = numbers[0]
smallest = numbers[0]

for num in numbers:
    if num > largest:
        largest = num
    if num < smallest:
        smallest = num

print(f"Largest: {largest}")
print(f"Smallest: {smallest}")
```

Exercise 6: Dictionaries

Question: Create a program that counts how many times each word appears in a sentence.

Solution:

```
sentence = "the quick brown fox jumps over the lazy dog the fox runs"
words = sentence.split()

word_count = {}

for word in words:
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1

for word, count in word_count.items():
    print(f"{word}: {count}")
```

Exercise 7: Functions

Question: Write a function that checks if a number is prime.

Solution:

```
def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False
    return True

# Test it
for n in range(2, 20):
    if is_prime(n):
        print(f"{n} is prime")
```

Exercise 8: OOP

Question: Create a Rectangle class with methods to calculate area and perimeter.

Solution:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

rect = Rectangle(5, 3)
print(f"Area: {rect.area()}")
print(f"Perimeter: {rect.perimeter()}")
```

Exercise 9: File Handling

Question: Write a program that counts the number of lines, words, and characters in a text file.

Solution:


```
with open("sample.txt", "r") as file:
    content = file.read()

lines = content.split("\n")
words = content.split()
characters = len(content)

print(f"Lines: {len(lines)}")
print(f"Words: {len(words)}")
print(f"Characters: {characters}")
```

Exercise 10: Exception Handling

Question: Write a program that keeps asking the user for a number until they enter a valid integer.

Solution:

```
while True:
    try:
        number = int(input("Enter a whole number: "))
        print(f"You entered: {number}")
        break
    except ValueError:
        print("That's not a valid whole number. Try again.")
```

Keep practicing! The more small problems you solve, the more natural Python will feel. Try modifying these exercises — change the rules, add new conditions, or combine ideas from different chapters into one program.

Chapter 20: Important Questions and Answers — Basics

This chapter covers frequently asked questions on Python fundamentals — useful for exams, interviews, and self-testing.

Q1. What is Python? What are its key features?

Answer: Python is a high-level, interpreted, general-purpose programming language. Key features include:

- **Easy to read and write** — syntax resembles plain English.
- **Interpreted** — code runs line by line, no separate compilation step needed.
- **Dynamically typed** — no need to declare variable types.
- **Object-oriented** — supports classes, objects, inheritance.
- **Extensive standard library** — “batteries included” philosophy.
- **Cross-platform** — runs on Windows, Mac, Linux without changes.
- **Large ecosystem** — huge number of third-party packages via pip.

Q2. What is the difference between Python 2 and Python 3?

Answer: Python 2 is no longer maintained (officially retired in 2020). Key differences:

Feature	Python 2	Python 3
print statement	<code>print "hello"</code>	<code>print("hello")</code>
Division	5/2 gives 2	5/2 gives 2.5
String type	ASCII by default	Unicode by default
Support	Discontinued	Actively maintained

All new projects should use Python 3.

Q3. Is Python compiled or interpreted?

Answer: Python is primarily an **interpreted** language. The interpreter reads and executes code line by line. Internally, Python source code is first compiled into byte-code (.pyc files), which is then run by the Python Virtual Machine (PVM) — so it’s technically a mix, but from a user’s perspective it behaves as interpreted.

Q4. What is PEP 8?

Answer: PEP 8 is Python’s official style guide. It defines conventions for writing readable code: 4 spaces for indentation, meaningful variable names, limiting line length to 79 characters, spacing around operators, and more. Following PEP 8 makes code easier for other developers (and your future self) to read.

Q5. What is the difference between `is` and `==`?

Answer: - `==` checks if two variables have the **equal value**. - `is` checks if two variables point to the **same object in memory**.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)    # True (same values)
print(a is b)    # False (different objects in memory)
print(a is c)    # True (c points to the same object as a)
```

Q6. What are mutable and immutable data types?

Answer: - **Mutable** objects can be changed after creation: list, dict, set. - **Immutable** objects cannot be changed after creation: int, float, str, tuple, bool.

```
# Immutable example
s = "hello"
# s[0] = "H"    # Error! Strings can't be modified in place

# Mutable example
my_list = [1, 2, 3]
my_list[0] = 100    # Works fine
print(my_list)      # [100, 2, 3]
```

Q7. What is dynamic typing?

Answer: In Python, you don't need to declare a variable's type — it's determined automatically at runtime, and a variable can even be reassigned to a different type.

```
x = 10    # x is an int
x = "hello"    # now x is a str — totally legal in Python
```

Q8. What is the difference between list and tuple?

Answer:

Feature	List	Tuple
Mutability	Mutable (can change)	Immutable (cannot change)
Syntax	[1, 2, 3]	(1, 2, 3)
Performance	Slower	Faster
Use case	Data that changes	Data that should stay constant

Q9. What is the difference between `append()` and `extend()` in lists?

Answer:

```
list1 = [1, 2, 3]

list1.append([4, 5])
print(list1)    # [1, 2, 3, [4, 5]] -> adds the whole list as one item

list2 = [1, 2, 3]
list2.extend([4, 5])
print(list2)    # [1, 2, 3, 4, 5] -> adds each item individually
```

Q10. What is the difference between shallow copy and deep copy?

Answer: - **Shallow copy** creates a new object, but nested objects inside are still shared (references). - **Deep copy** creates a completely independent copy, including nested objects.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 100

print(shallow)    # [[100, 2], [3, 4]] -> changed! (shares inner lists)
print(deep)       # [[1, 2], [3, 4]] -> unaffected (fully independent)
```

Chapter 21: Important Questions and Answers — Control Flow and Loops

Q1. What is the difference between break, continue, and pass?

Answer: - break — exits the loop entirely. - continue — skips the current iteration and moves to the next. - pass — does nothing; it's a placeholder when syntax requires a statement.

```
for i in range(5):
    if i == 2:
        break          # stops the loop at i=2
    print(i)
# Output: 0 1

for i in range(5):
    if i == 2:
        continue      # skips printing 2
    print(i)
# Output: 0 1 3 4

for i in range(3):
    pass              # does nothing, but is syntactically valid
```

Q2. Why does Python use indentation instead of curly braces?

Answer: Python's design philosophy emphasizes readability. Forcing indentation to define code blocks (instead of optional {}) ensures that code is always visually structured the same way it executes, reducing bugs caused by misleading formatting.

Q3. What is the difference between for and while loops?

Answer: - for loop is used when you know the number of iterations or are iterating over a sequence (list, string, range). - while loop is used when the number of iterations is not known in advance and depends on a condition.

```
# for - known iterations
for i in range(5):
    print(i)

# while - condition-based
count = 0
while count < 5:
    print(count)
    count += 1
```

Q4. Can a while loop have an else block? What does it do?

Answer: Yes. The else block after a loop runs only if the loop completes **without** hitting a break.

```
n = 7
i = 2
while i < n:
    if n % i == 0:
        print(f"{n} is not prime")
        break
    i += 1
else:
    print(f"{n} is prime")
```

Q5. What does the range() function do, and what are its parameters?

Answer: range() generates a sequence of numbers. It takes up to 3 parameters: range(start, stop, step).

```
range(5)          # 0, 1, 2, 3, 4 (stop only)
range(2, 8)       # 2, 3, 4, 5, 6, 7 (start, stop)
range(0, 10, 2)   # 0, 2, 4, 6, 8 (start, stop, step)
range(10, 0, -1)  # 10, 9, 8, ..., 1 (counting backward)
```

Q6. How do you loop through a list with both index and value?

Answer: Use enumerate().

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(index, fruit)
# 0 apple
# 1 banana
# 2 cherry
```

Q7. What is a nested loop? Give an example.

Answer: A nested loop is a loop inside another loop. The inner loop completes all its iterations for each single iteration of the outer loop.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i*j}")
```

Q8. What will be the output of this code, and why?

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print(i, j)
```

Answer: Output:

```
0 0
1 0
2 0
```

Explanation: The inner loop breaks as soon as `j == 1`, so only `j = 0` ever gets printed for each value of `i`. `break` only exits the **inner** loop, not the outer one.

Chapter 22: Important Questions and Answers — Functions

Q1. What is the difference between a parameter and an argument?

Answer: A **parameter** is the variable listed in a function's definition. An **argument** is the actual value passed when calling the function.

```
def greet(name):    # 'name' is a parameter
    print(f"Hello, {name}")

greet("Kavi")       # "Kavi" is an argument
```

Q2. What is the difference between *args and **kwargs?

Answer: - *args collects extra **positional** arguments into a tuple. - **kwargs collects extra **keyword** arguments into a dictionary.

```
def example(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)

example(1, 2, 3, name="Kavi", age=22)
# args: (1, 2, 3)
# kwargs: {'name': 'Kavi', 'age': 22}
```

Q3. What is a recursive function? Give an example.

Answer: A recursive function is a function that calls itself to solve a smaller version of the same problem, until it reaches a base case.

```
def factorial(n):
    if n == 0 or n == 1:    # base case
        return 1
    return n * factorial(n - 1)    # recursive case

print(factorial(5))    # 120
```

Note: Every recursive function needs a base case to avoid infinite recursion (which causes a RecursionError).

Q4. What is the difference between return and print() inside a function?

Answer: - return sends a value back to the caller, which can be stored or used further. - print() only displays output on the screen; the value is not stored or usable elsewhere.


```
def add_return(a, b):
    return a + b

def add_print(a, b):
    print(a + b)

result = add_return(3, 4)    # result = 7, usable later
add_print(3, 4)             # prints 7, but nothing is returned (None)
```

Q5. What is a default argument? What is a common pitfall?

Answer: A default argument provides a fallback value if the caller doesn't supply one.

```
def greet(name="Guest"):
    print(f"Hello, {name}")

greet()          # Hello, Guest
greet("Arun")    # Hello, Arun
```

Pitfall: Never use a mutable object (like a list) as a default argument — it's shared across all calls.

```
def add_item(item, items=[]):    # BAD PRACTICE
    items.append(item)
    return items

print(add_item("a"))    # ['a']
print(add_item("b"))    # ['a', 'b'] <- unexpected! Same list reused.
```

Fix: use None as default and create a new list inside the function.

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Q6. What is variable scope? Explain local, global, and the global keyword.

Answer: Scope determines where a variable can be accessed.

- **Local scope:** variables defined inside a function, accessible only within that function.
- **Global scope:** variables defined outside any function, accessible everywhere.

```
x = 10    # global
```

```
def show():
```

```

x = 5    # local, separate from global x
print(x)

show()    # 5
print(x)  # 10

```

To modify a global variable from inside a function, use `global`:

```

counter = 0

def increment():
    global counter
    counter += 1

increment()
print(counter)    # 1

```

Q7. What is a lambda function? When would you use one?

Answer: A lambda is a small, anonymous, one-expression function, typically used for short operations passed to other functions like `map()`, `filter()`, or `sorted()`.

```

students = [("Kavi", 85), ("Arun", 92), ("Priya", 78)]

# Sort by score (second item in each tuple)
sorted_students = sorted(students, key=lambda s: s[1], reverse=True)
print(sorted_students)

```

Q8. What is the difference between `map()`, `filter()`, and `reduce()`?

Answer: - `map()` applies a function to every item in an iterable. - `filter()` keeps only items that satisfy a condition. - `reduce()` (from `functools`) combines all items into a single value.

```

from functools import reduce

numbers = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, numbers))
print(squared)    # [1, 4, 9, 16, 25]

evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)      # [2, 4]

total = reduce(lambda a, b: a + b, numbers)
print(total)      # 15

```

Chapter 23: Important Questions and Answers — OOP

Q1. What are the four pillars of OOP?

Answer: 1. **Encapsulation** — bundling data and methods together, restricting direct access. 2. **Inheritance** — a class reusing properties and methods from another class. 3. **Polymorphism** — the same method name behaving differently across classes. 4. **Abstraction** — hiding internal implementation, exposing only what's needed.

Q2. What is the difference between a class and an object?

Answer: A **class** is a blueprint/template. An **object** is a specific instance created from that blueprint, with its own data.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car1 = Car("Toyota")    # car1 is an object of class Car
car2 = Car("Honda")     # car2 is another, separate object
```

Q3. What is self in Python classes?

Answer: self refers to the current instance of the class. It's how a method accesses the object's own attributes and other methods. It must be the first parameter of every instance method (Python passes it automatically when you call the method).

```
class Dog:
    def __init__(self, name):
        self.name = name        # self.name belongs to this specific object

    def bark(self):
        print(f"{self.name} says Woof!")    # self.name refers to the calling object's name
```

Q4. What is the difference between instance variables and class variables?

Answer: - **Instance variables** are unique to each object, defined with self.. - **Class variables** are shared across all instances of the class.

```
class Student:
    school_name = "Green Valley School"    # class variable, shared

    def __init__(self, name):
        self.name = name                  # instance variable, unique

s1 = Student("Kavi")
s2 = Student("Arun")
```

```
print(s1.school_name)    # Green Valley School
print(s2.school_name)    # Green Valley School (same for all objects)
print(s1.name)           # Kavi (unique to s1)
```

Q5. What is method overriding? Give an example.

Answer: Method overriding happens when a child class defines a method with the same name as one in its parent class, replacing the parent's behavior.

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):           # overrides Animal's speak()
        print("Dog barks")

d = Dog()
d.speak()    # Dog barks
```

Q6. What does super() do?

Answer: super() allows a child class to call methods (often __init__) from its parent class, useful when extending rather than fully replacing parent behavior.

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # runs Animal's __init__
        self.breed = breed

d = Dog("Rocky", "Labrador")
print(d.name, d.breed)    # Rocky Labrador
```

Q7. What is the difference between single, multiple, and multilevel inheritance?

Answer: - **Single inheritance:** one child class inherits from one parent. - **Multiple inheritance:** one child class inherits from more than one parent. - **Multilevel inheritance:** a chain — class C inherits from B, which inherits from A.

```
# Single
class A: pass
class B(A): pass
```

```
# Multiple
class C: pass
class D: pass
class E(C, D): pass

# Multilevel
class F: pass
class G(F): pass
class H(G): pass
```

Q8. What is encapsulation? How is it implemented in Python?

Answer: Encapsulation restricts direct access to an object's internal data, exposing controlled access through methods. Python uses naming conventions:

- `_variable` — a single underscore signals “internal use, please don't touch directly” (convention only, not enforced).
- `__variable` — a double underscore triggers name mangling, making it harder (but not impossible) to access from outside.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance    # "private" by convention

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = Account(1000)
acc.deposit(500)
print(acc.get_balance())    # 1500
# print(acc.__balance)      # AttributeError - not directly accessible
```

Q9. What is polymorphism? Give a practical example.

Answer: Polymorphism means the same interface (method name) can behave differently depending on the object calling it.

```
class Cat:
    def speak(self):
        return "Meow"

class Dog:
    def speak(self):
        return "Woof"
```

```
animals = [Cat(), Dog()]

for animal in animals:
    print(animal.speak())    # Meow, then Woof – same method name, different behavior
```

Q10. What is the difference between an abstract class and a regular class?

Answer: An abstract class cannot be instantiated directly — it's meant to be a template that other classes inherit from and implement. Python provides this via the abc module.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

# shape = Shape()    # Error! Cannot instantiate abstract class
circle = Circle(5)
print(circle.area())    # 78.5
```

Chapter 24: Important Questions and Answers — Data Structures

Q1. What is the difference between a list, tuple, set, and dictionary?

Answer:

Feature	List	Tuple	Set	Dictionary
Ordered	Yes	Yes	No	Yes (insertion order, Python 3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique, values can repeat
Syntax	[1, 2]	(1, 2)	{1, 2}	{"a": 1}

Q2. How do you remove duplicates from a list?

Answer: Convert it to a set (which only stores unique values), then back to a list if order doesn't matter much.

```
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(numbers))
print(unique)    # [1, 2, 3, 4] (order may change)
```

To preserve order while removing duplicates:

```
numbers = [1, 2, 2, 3, 3, 3, 4]
seen = []
for num in numbers:
    if num not in seen:
        seen.append(num)
print(seen)    # [1, 2, 3, 4] (order preserved)
```

Q3. How do you merge two dictionaries?

Answer: In Python 3.9+, you can use the | operator. Older versions can use update() or ** unpacking.

```
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}

merged = dict1 | dict2          # Python 3.9+
print(merged)                  # {'a': 1, 'b': 3, 'c': 4}

merged2 = {**dict1, **dict2}    # works in older versions too
print(merged2)                 # {'a': 1, 'b': 3, 'c': 4}
```

Note: when keys overlap, the second dictionary's value wins.

Q4. What is list slicing? Give examples.

Answer: Slicing extracts a portion of a list using `list[start:stop:step]`.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5])      # [2, 3, 4]
print(numbers[:4])      # [0, 1, 2, 3]
print(numbers[6:])      # [6, 7, 8, 9]
print(numbers[::2])      # [0, 2, 4, 6, 8] (every 2nd item)
print(numbers[::-1])     # reversed list
```

Q5. What is a set, and what operations does it support?

Answer: A set is an unordered collection of unique elements, supporting mathematical set operations.

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b))        # {1, 2, 3, 4, 5, 6}
print(a.intersection(b)) # {3, 4}
print(a.difference(b))   # {1, 2}
print(a.symmetric_difference(b)) # {1, 2, 5, 6}
```

Q6. How do you sort a list of dictionaries by a specific key?

Answer: Use `sorted()` with a `key` parameter (often a `lambda`).

```
students = [
    {"name": "Kavi", "score": 85},
    {"name": "Arun", "score": 92},
    {"name": "Priya", "score": 78},
]

sorted_students = sorted(students, key=lambda s: s["score"], reverse=True)
for s in sorted_students:
    print(s["name"], s["score"])
```

Q7. What is the time complexity of common list and dictionary operations?

Answer:

Operation	List	Dictionary
Access by index/key	$O(1)$	$O(1)$
Search by value	$O(n)$	$O(1)$ for key lookup
Insert at end	$O(1)$	$O(1)$
Insert at beginning	$O(n)$	N/A
Delete	$O(n)$	$O(1)$

Dictionaries are generally faster for lookups because they use hashing internally.

Chapter 25: Important Questions and Answers — Exceptions and Modules

Q1. What is the difference between an error and an exception?

Answer: An **error** is a serious problem that often can't be handled (like a `SyntaxError` — code that doesn't even run). An **exception** is an issue that occurs during execution, which **can** be caught and handled gracefully using try-except.

Q2. What is the purpose of finally in exception handling?

Answer: The finally block always executes, whether or not an exception occurred — commonly used for cleanup actions like closing files or releasing resources.

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    print("This always runs")
    # file.close() would typically go here if not using 'with'
```

Q3. What is the difference between except Exception and a specific exception like except ValueError?

Answer: `except Exception` catches almost **any** error, which can hide bugs you didn't anticipate. Catching specific exceptions (`ValueError`, `TypeError`, etc.) is better practice because it handles only the errors you expect, letting unexpected ones surface clearly.

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid number format")
except Exception as e:
    print(f"Unexpected error: {e}")
```

specific - only catches conversion errors
general - catches anything else

Q4. Can you have multiple except blocks for one try? Show an example.

Answer: Yes, Python checks each except block in order and runs the first one that matches.

```
try:
    result = 10 / int(input("Enter a number: "))
except ValueError:
    print("That's not a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
else:
    print(f"Result: {result}")
```

Q5. How do you create and raise a custom exception?

Answer: Define a class that inherits from Exception, then use raise to trigger it.

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("Not enough balance for this withdrawal")
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print(f"Error: {e}")
```

Q6. What is the difference between a module and a package?

Answer: A **module** is a single Python file (.py) containing code. A **package** is a folder containing multiple modules, along with an `__init__.py` file that marks it as a package.

```
my_package/
├── __init__.py
├── module1.py
└── module2.py
```

```
from my_package import module1
```

Q7. What is the difference between import module and from module import function?

Answer:

```
import math
print(math.sqrt(16))      # must prefix with module name

from math import sqrt
print(sqrt(16))           # can call directly, no prefix needed
```

The first keeps the module's namespace separate (clearer where functions come from); the second is more convenient for frequently used functions but can cause naming conflicts if you import from multiple modules.

Q8. What is pip, and how do you use it?

Answer: pip is Python's package manager, used to install third-party libraries from the Python Package Index (PyPI).

```
pip install requests          # install a package
pip uninstall requests        # remove a package
pip list                      # list installed packages
pip freeze > requirements.txt # save installed packages to a file
pip install -r requirements.txt # install from a requirements file
```

Chapter 26: Frequently Asked Interview-Style Questions

Q1. How is memory managed in Python?

Answer: Python uses automatic memory management with a private heap. Objects are allocated on this heap, and Python's built-in **garbage collector** reclaims memory from objects no longer referenced anywhere (mainly using reference counting, with a cyclic garbage collector for objects that reference each other in cycles).

Q2. What is the Global Interpreter Lock (GIL)?

Answer: The GIL is a mutex that allows only one thread to execute Python bytecode at a time, even on multi-core systems. It simplifies memory management but limits true parallelism in multi-threaded Python programs (CPU-bound tasks). Workarounds include using multiprocessing (separate processes instead of threads) or libraries that release the GIL for heavy computation.

Q3. What are Python's built-in data types?

Answer: Numeric (int, float, complex), Sequence (str, list, tuple, range), Mapping (dict), Set types (set, frozenset), Boolean (bool), and None (NoneType).

Q4. What is the difference between == and .equals()-style comparisons across types?

Answer: Python doesn't have a separate .equals() method like Java — == is used for value comparison universally, and Python lets you customize this behavior for custom classes by overriding the __eq__ method.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1 == p2)    # True, because __eq__ is defined
```

Q5. What is a generator, and how is it different from a normal function?

Answer: A generator uses yield instead of return, producing values one at a time and pausing its state between each call — making it memory-efficient for large or infinite sequences, since it doesn't store all values in memory at once.

```
def count_up(n):
    i = 1
    while i <= n:
        yield i
        i += 1

gen = count_up(5)
print(next(gen))    # 1
print(next(gen))    # 2
# A normal function would compute and return everything at once;
# a generator computes only when asked.
```

Q6. What is a decorator?

Answer: A decorator is a function that wraps another function to extend or modify its behavior without changing its source code.

```
def timer_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@timer_decorator
def say_hello():
    print("Hello!")

say_hello()
# Calling say_hello
# Hello!
# Finished say_hello
```

Q7. What is the difference between deepcopy and assignment (=)?

Answer: Assignment (=) just creates a new reference to the **same object** — no copying happens at all.

```
a = [1, 2, 3]
b = a           # b is just another name for the same list
b.append(4)
print(a)        # [1, 2, 3, 4] -> a changed too!

import copy
c = copy.deepcopy(a) # c is a fully independent copy
c.append(5)
print(a)        # unaffected
```

Q8. What is duck typing in Python?

Answer: “If it walks like a duck and quacks like a duck, it’s a duck.” Python doesn’t check an object’s specific type before using it — it only checks whether the object supports the required behavior (methods/attributes).

```
class Duck:
    def sound(self):
        return "Quack"

class Dog:
    def sound(self):
        return "Woof"

def make_sound(animal):
    print(animal.sound())    # works for any object with a sound() method

make_sound(Duck())    # Quack
make_sound(Dog())    # Woof
```

Q9. What is the difference between `__init__` and `__new__`?

Answer: `__new__` creates a new instance of a class (allocates memory) and is called first; `__init__` initializes that instance’s attributes afterward. In most everyday code, you only need to define `__init__` — `__new__` is rarely overridden except in advanced scenarios.

Q10. What does the `__name__ == "__main__"` check do?

Answer: It checks whether a script is being run directly, versus being imported as a module into another script. Code inside this block only runs when the file is executed directly.

```
def main():
    print("Running directly")

if __name__ == "__main__":
    main()
```

If this file is imported elsewhere (`import myfile`), `main()` will **not** run automatically — only when the file itself is executed.

Chapter 27: Rapid-Fire Question Bank (One-Line Answers)

Use these for quick revision before an exam or interview. Try answering each one yourself before reading the answer.

- 1. What does `len()` do?** Returns the number of items in a sequence (string, list, tuple, dict, set).
- 2. What is the output of `2 ** 3`?** 8 — the `**` operator means exponentiation.
- 3. What is the output of `7 // 2`?** 3 — `//` performs floor (integer) division.
- 4. How do you check the type of a variable?** Using the `type()` function, e.g. `type(x)`.
- 5. What does `None` represent?** The absence of a value — Python’s equivalent of “null” in other languages.
- 6. How do you create an empty list, dictionary, and set?** `[]` for list, `{}` for dictionary, `set()` for set (note: `{}` alone creates a dict, not a set).
- 7. What does the `in` keyword do?** Checks membership — whether a value exists in a sequence. E.g. `"a" in "cat"` returns `True`.
- 8. How do you convert a string to a list of characters?** `list("hello")` gives `['h', 'e', 'l', 'l', 'o']`.
- 9. What is string concatenation?** Joining two or more strings using `+`, e.g. `"Hello" + " " + "World"`.
- 10. What does `str.join()` do?** Joins elements of an iterable into a single string, using the string as a separator.
- 11. What is an f-string?** A formatted string literal (`f"{variable}"`) that lets you embed expressions directly inside string literals.
- 12. What does the `not` operator do?** Reverses a boolean value — `not True` is `False`.
- 13. What is the difference between `and` and `or`?** `and` requires both conditions to be `True`; `or` requires at least one to be `True`.
- 14. What does `id()` return?** The unique memory address (identity) of an object.
- 15. How do you check if a key exists in a dictionary?** `if key in my_dict:`
- 16. What is the output of `bool(0)` and `bool("")`?** Both are `False` — `0`, `""`, `None`, `[]`, `{}`, and `()` are all “falsy” in Python.
- 17. What does `type([]) == list` evaluate to?** `True`.
- 18. What is an iterable?** Any object that can be looped over — lists, tuples, strings, dictionaries, sets, and more.
- 19. What is an iterator?** An object that produces items one at a time, supporting the `next()` function, created from an iterable using `iter()`.
- 20. What does `zip()` do?** Combines multiple iterables element-wise into tuples. E.g. `zip([1,2], ['a','b'])` gives `[(1,'a'), (2,'b')]`.

- 21. What is list comprehension syntax?** [expression for item in iterable if condition]
- 22. What does the del keyword do?** Deletes a variable, list item, or dictionary key.
- 23. How do you check Python's version?** `python --version` in the terminal, or `import sys; print(sys.version)` in code.
- 24. What does input() always return?** A string — you must convert it manually if you need a number.
- 25. What is the difference between / and //?** / gives a float result (true division); // gives an integer result (floor division).
- 26. What does str(), int(), and float() do?** Convert a value to a string, integer, or floating-point number, respectively.
- 27. What is an f-string equivalent using .format()?** `"Hello {}".format(name)` is equivalent to `f"Hello {name}"`.
- 28. What does assert do?** Tests if a condition is True; raises an `AssertionError` if it's False. Often used for debugging.
- 29. What is the purpose of __init__.py in a folder?** Marks the folder as a Python package, allowing its modules to be imported.
- 30. What does with open(...) as f: do differently from open(...)?** Automatically closes the file when the block ends, even if an error occurs — safer than manually calling `.close()`.
- 31. What is the output of "abc" * 3?** `"abccabccabcc"` — strings can be repeated using `*`.
- 32. What does list.sort() return?** None — it sorts the list in place and does not return a new list (unlike `sorted()`).
- 33. How do you reverse a string?** `my_string[::-1]`
- 34. What is a docstring?** A string literal placed right after a function/class/module definition to document its purpose, written with triple quotes `"""like this"""`.
- 35. What does ** mean in def func(**kwargs)?** It collects extra keyword arguments into a dictionary.
- 36. What is the difference between is None and == None?** `is None` is the recommended way (checks identity, faster and more reliable); `== None` works too but is considered less idiomatic.
- 37. How do you check if a number is even or odd?** `if number % 2 == 0:` for even, else odd.
- 38. What does try / except / else do differently from just try / except?** The `else` block runs only if no exception occurred in the `try` block.
- 39. What is the purpose of continue in a loop?** Skips the rest of the current iteration and moves to the next one.

40. What does `enumerate(list, start=1)` do? Loops through a list with index and value, but starts counting from 1 instead of the default 0.

Chapter 28: Where to Go From Here

Congratulations on making it through this guide! Here are some good next steps to keep building your Python skills:

1. **Practice daily** — even 20-30 minutes a day builds strong habits.
2. **Build small projects** — a to-do list app, a calculator, a quiz game.
3. **Learn a framework** — Flask or Django for web development, Pandas for data analysis.
4. **Read other people's code** — explore open-source projects on GitHub.
5. **Solve coding problems** — try sites like HackerRank, LeetCode, or Codewars.
6. **Join a community** — Python forums, Discord servers, or local meetups.

Quick Reference: Common Built-in Functions

Function	Purpose
<code>print()</code>	Display output
<code>input()</code>	Get user input
<code>len()</code>	Get length of a list/string/etc
<code>type()</code>	Check the data type
<code>int(), float(), str()</code>	Convert between types
<code>range()</code>	Generate a sequence of numbers
<code>sorted()</code>	Return a sorted list
<code>sum()</code>	Add up numbers in a list
<code>max(), min()</code>	Largest / smallest value
<code>enumerate()</code>	Loop with index and value
<code>zip()</code>	Combine two lists together

```
names = ["Arun", "Kavi", "Priya"]
ages = [21, 22, 23]

for name, age in zip(names, ages):
    print(name, age)
```

Happy coding!